Partial Reproduction of Mimicry Attacks Against Pagoda

CS 790 – Group 4 (Networking)
Team Members: Chijioke Okogwu and David
Instructor: Professor Zhen Zeng

1. Project Objective

Our project evaluates whether Pagoda, a provenance-graph host intrusion detection system, detects original attack graphs differently from mimicry-evasion graphs. We partially reproduce the Pagoda experiment reported by Goyal et al. [1], using the researchers' released implementation and graph datasets [2]. Rather than rebuilding Pagoda or generating new attacks, we execute the released detector on selected benign, attack, and pre-generated evasion graphs. We also extend our evaluation to previously unseen benign graphs to examine false alarms.

2. Methodology and Progress

We configured the project on Windows 11 using Python 3.11.9, a virtual environment, PyTorch on CPU (no CUDA), and 16 GB RAM. We extracted the released CSV datasets and wrote scripts to prepare reproducible experiment folders. The available dataset includes 71 benign training graphs, a separate benign test split, and released attack and evasion graphs. We left Pagoda's detection logic unchanged and kept saved artifacts separate across runs.

Our experiments proceeded in five stages: (1) a one-graph pilot to verify execution; (2) a five-benign-graph training configuration with five attack and five evasion graphs; (3) a 71-benign-graph training configuration using the *same* attack and evasion samples; and (4–5) held-out benign evaluations using the saved artifacts from each configuration. For the held-out tests, we selected five unseen benign graphs (ADM-370, ADM-371, ADM-372, ADM-373, and ADM-375) and applied our evaluation script to Pagoda's saved frequency database and thresholds.

We inspected the released pipeline, including frequency-database construction, path extraction, scoring, threshold selection, and statistics reporting. A key implementation detail is that the printed values `fpr`, `tpr`, and `evasion` are *counts*, not percentages. We also resolved dependency and data-extraction issues, handled intermediate artifact locations, and checked runtime on our CPU-only environment. Cached reruns took approximately 27 seconds for the 71-graph detector run and 31 seconds for the held-out evaluation; these are not fresh-training benchmarks.

3. Results

Table 1 summarizes our main results. The one-graph pilot completed successfully, detecting both its original attack and evasion graph; its primary purpose was to validate the pipeline.

Table 1: Results for the same five attack, evasion, and held-out benign samples under two training configurations.

Metric	**5 benign train**	**71 benign train**
Original attacks detected	5/5	5/5
Evasion graphs detected	5/5	0/5
Evasion graphs escaping detection	0/5	5/5
Training benign graphs flagged	0/5	0/71
Held-out benign graphs falsely flagged	5/5	5/5
Graph anomaly threshold (approx.)	0.1038	0.1237

With five benign training graphs, Pagoda detected all five original attacks and all five evasion graphs. With 71 benign training graphs, it still detected all five original attacks but detected none of the five evasion graphs. Thus, the evasion success rate in our selected sample changed from 0/5 to 5/5. This is a *qualitative partial reproduction* of the mimicry-evasion behavior studied by Goyal et al. [1], not a reproduction of their full-scale quantitative evaluation.

The graph threshold increased from approximately 0.1038 to 0.1237 between configurations, while the path threshold remained near 1.0×10^{-4}. In our additional held-out test, all five unseen benign graphs were flagged under both configurations, by the graph-level check rather than the path-level check. Their graph scores under the 71-graph configuration were approximately 0.443–0.470, exceeding its 0.1237 threshold.

4. Discussion and Limitations

Our main observation is that changing the amount of benign training data coincided with a reversal in the five evasion verdicts, even though the attack and evasion samples were held constant. The higher graph threshold is a plausible contributor, but the learned frequency database also changes. Without logging and comparing each evasion graph's score under both configurations, we cannot attribute the reversal to the threshold alone.

The held-out evaluation revealed a second concern: all five selected unseen benign graphs were flagged. This is a 100% false-positive rate *within our five-graph custom test only*; it is not an estimate of Pagoda's general false-positive rate. The discrepancy between zero training-benign graphs flagged and five held-out graphs flagged shows why in-sample counts should not be used as evidence of generalization. Our evaluation procedure may differ from the original experimental protocol and warrants further validation.

Several limitations remain. We evaluated only five original attacks, five released evasion graphs, and five held-out benign graphs. We did not generate the mimicry graphs ourselves, verify one-to-one correspondence between selected attack and evasion files, or measure the number of added events per matched pair. Furthermore, inspection of the released threshold-calibration code indicated that attack scores contribute to calibration, so the attack detection measurements are not fully independent of threshold selection. Finally, cached rerun times do not represent a clean end-to-end reproduction from scratch.

Another limitation was our available computing hardware. Our experiments were conducted on a Windows 11 computer with 16 GB of RAM, using Python 3.11.9 and a CPU-only installation of PyTorch because GPU acceleration was unavailable. These resource constraints influenced our decision to evaluate five attack graphs and five mimicry-evasion graphs rather than the full set of 100 graphs in each category. Although we successfully processed all 71 benign training graphs, we did not benchmark the complete 100-graph evaluation. Consequently, our experiments demonstrate the feasibility of a smaller-scale reproduction but do not establish the computational performance or detection outcomes of the full evaluation.

Overall, we successfully configured and executed the released Pagoda pipeline, reproduced mimicry-evasion behavior on a small selected sample, and added an independent benign-data check that exposed an important evaluation limitation. Next steps would be to log per-graph scores, validate attack/evasion pairing, and extend the held-out evaluation to the remaining benign test graphs.

References

[1] A. Goyal et al., "Sometimes, You Aren't What You Do: Mimicry Attacks against Provenance Graph Host Intrusion Detection Systems," *Network and Distributed System Security Symposium (NDSS)*, 2023. https://www.ndss-symposium.org/ndss-paper/sometimes-you-arent-what-you-do-mimicry-attacks-against-provenance-graph-host-intrusion-detection-syst

[2] STS Lab, *Mimicry Provenance Generator*, original research implementation and datasets. https://bitbucket.org/sts-lab/mimicry-provenance-generator

[3] CS 790 Group 4, *Mimicry Attack Reproduction*, experimental scripts and results, 2026. https://github.com/Williamokogwu/cs790-mimicry-reproduction